SQL Practice Queries

Lesson 4 - Aggregate Functions, GROUP BY, HAVING & Advanced ORDER BY

Prepared for DTank54 Group Students
Oracle HR Schema - A1 English Level

Topics Covered

ORDER BY (Advanced Sorting)	Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
GROUP BY (Single & Multiple Columns)	HAVING (Filtering Groups)

Important Notes Before You Start

> *In this lesson, you will learn how to analyze data in groups. Aggregate functions like COUNT, SUM, AVG, MIN, and MAX let you calculate statistics across many rows. GROUP BY organizes rows into groups. HAVING filters these groups after aggregation. These tools are the foundation of data analysis and business reporting in SQL.*

Section 1: Advanced ORDER BY

You already know the basics of ORDER BY from previous lessons. In this section, we will learn more advanced ways to sort data. You can sort by column position number, by an expression (like a calculation), by a column alias, and even control how NULL values are sorted. These techniques are very useful when you write complex reports.

1.1 ORDER BY with Column Position

Instead of writing the column name, you can use the column position number in the SELECT list. The first column is 1, the second column is 2, and so on. This can save time when column names are very long. However, using column names is usually clearer and safer. If you change the SELECT list, the position numbers might point to wrong columns.

```
SELECT first_name, last_name, salary, department_id
FROM employees
ORDER BY 3 DESC;
```

What this query does: This sorts employees by salary from high to low. The number 3 refers to the third column in SELECT, which is salary. ORDER BY 3 DESC is the same as ORDER BY salary DESC. This is shorter to write when you know the column position.

```
SELECT department_id, job_id, first_name, last_name, salary
FROM employees
ORDER BY 1, 5 DESC;
```

What this query does: First sorts by department_id (column 1) in ascending order. Inside each department, sorts by salary (column 5) from high to low. This is the same as ORDER BY department_id, salary DESC. Using position numbers can make the query shorter when you have many columns.

1.2 ORDER BY with Expressions

You can sort by a mathematical expression or calculation, not just by a column. For example, you can sort by salary * 12 (annual salary) or by salary + commission. This is very useful when you want to rank employees by a calculated value that does not exist as a

column in the table.

```
SELECT first_name, last_name, salary, salary * 12 AS annual_salary
FROM employees
ORDER BY salary * 12 DESC;
```

What this query does: Calculates annual salary (salary * 12) for each employee and sorts by this calculated value from high to low. Even though annual_salary is not a real column in the table, ORDER BY can use any valid expression. You can also write ORDER BY annual_salary DESC because Oracle lets you use the alias in ORDER BY.

```
SELECT first_name, last_name, salary, commission_pct,
salary + salary * commission_pct AS total_pay
FROM employees
WHERE commission_pct IS NOT NULL
ORDER BY salary + salary * commission_pct DESC;
```

What this query does: Shows only employees who have a commission (commission_pct IS NOT NULL). It calculates total pay (salary plus commission) and sorts by this total from high to low. The expression salary + salary * commission_pct is used both in SELECT and in ORDER BY. Remember: if commission_pct is NULL, the calculation returns NULL. That is why we use WHERE IS NOT NULL.

1.3 ORDER BY with Column Aliases

Oracle allows you to use column aliases in ORDER BY. This makes your queries more readable. Instead of repeating a long expression, you can give it a short alias in SELECT and use that alias in ORDER BY. This is a very common and recommended practice.

```
SELECT first_name, last_name,
salary * 12 + salary * NVL(commission_pct, 0) AS total_annual
FROM employees
ORDER BY total_annual DESC;
```

What this query does: Calculates total annual pay including commission. The NVL function replaces NULL commission with 0, so the math works for all employees. The result column is named total_annual. ORDER BY uses this alias instead of repeating the full expression. NVL is a very important function that we will use many times in this lesson.

1.4 NULLS FIRST / NULLS LAST

By default, Oracle sorts NULL values as the highest values. This means NULLs appear at the end for ascending order (ASC) and at the beginning for descending order (DESC). You can control this behavior with NULLS FIRST and NULLS LAST keywords. This is very important when you work with columns that have NULL values, like commission_pct.

```
SELECT first_name, last_name, commission_pct
FROM employees
ORDER BY commission_pct ASC NULLS LAST;
```

What this query does: Shows all employees sorted by commission_pct in ascending order. NULLS LAST means employees with no commission (NULL) appear at the very end of the result. Without NULLS LAST, Oracle puts NULL values last anyway in ASC order, so this is mainly for clarity. It becomes more important with DESC order.

```
SELECT first_name, last_name, commission_pct
FROM employees
ORDER BY commission_pct DESC NULLS FIRST;
```

What this query does: Sorts by commission_pct from high to low. NULLS FIRST puts employees without commission at the top of the list. Without NULLS FIRST, Oracle puts NULLs last in DESC order by default. This is useful when you want to see employees without commission first, then the ones with highest commission.

1.5 ORDER BY Case-Insensitive Sort

By default, ORDER BY sorts text in a case-sensitive way. This means uppercase letters come before lowercase letters. For example, "Z" comes before "a". If you want a case-insensitive sort, you can use UPPER() or LOWER() in ORDER BY. This ensures that names are sorted alphabetically regardless of their case.

```
SELECT first_name, last_name
FROM employees
ORDER BY UPPER(first_name);
```

What this query does: Sorts employees by first_name in alphabetical order, ignoring case. UPPER(first_name) converts all names to uppercase before sorting. This way, "adam" and "Adam" are treated as the same letter. Without UPPER(), Oracle sorts "Adam" before "adam" because uppercase "A" has a lower character code than lowercase "a".

Section 2: Aggregate Functions

Aggregate functions perform a calculation on a set of rows and return a single value. They are the most important tools for data analysis in SQL. For example, you can count how many employees work in a department, calculate the average salary, find the

highest or lowest salary, or calculate the total of all salaries. There are five main aggregate functions in SQL.

2.1 COUNT - Count Rows

COUNT is the most commonly used aggregate function. It counts the number of rows. There are three ways to use COUNT: COUNT(*) counts all rows including NULLs. COUNT(column) counts only rows where that column is NOT NULL. COUNT(DISTINCT column) counts unique non-NULL values. Each form is useful for different situations.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

What this query does: Counts ALL rows in the employees table. The star (*) means "all columns". COUNT(*) counts every row, even if some columns have NULL values. The result is a single number. This query tells you how many employees are in the company. The alias total_employees makes the result column name clear and readable.

```
SELECT COUNT(commission_pct) AS employees_with_commission
FROM employees;
```

What this query does: Counts only employees who HAVE a commission percentage. COUNT(commission_pct) ignores rows where commission_pct is NULL. In the HR schema, most employees do not have a commission (it is NULL for non-sales jobs). So this number will be much smaller than COUNT(*). The difference between COUNT(*) and COUNT(commission_pct) tells you how many employees have no commission.

```
SELECT COUNT(DISTINCT department_id) AS num_departments
FROM employees;
```

What this query does: Counts how many different (unique) departments have employees. DISTINCT removes duplicates before counting. If 50 employees work in department 90, department 90 is counted only once. This is useful for answering questions like "How many different departments do we have employees in?"

2.2 SUM - Total of Values

SUM adds up all the values in a numeric column. It only works with numbers. SUM ignores NULL values (it simply skips them). You can use SUM to calculate total salary, total sales, total quantity, and more. SUM is often used with GROUP BY to calculate totals for each group.

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

What this query does: Adds up all salary values in the employees table. The result is a single number representing the total monthly salary paid to all employees. This is useful for budget planning and financial reports. If you want the annual total, you can write SUM(salary * 12).

```
SELECT SUM(salary) AS total_it_salary
FROM employees
WHERE department_id = 60;
```

What this query does: Calculates the total salary for all employees in department 60 (IT department). SUM works with WHERE to calculate totals for a filtered set of rows. This helps answer questions like "How much do we pay the IT department in total each month?" You can combine SUM with any WHERE condition.

2.3 AVG - Average of Values

AVG calculates the arithmetic mean (average) of a numeric column. It divides the SUM by the COUNT of non-NULL values. AVG ignores NULL values automatically. This means AVG(salary) averages only employees who have a salary value. Be careful: if you use AVG with NULL values, the result might be different from what you expect because NULLs are excluded from both the sum and the count.

```
SELECT AVG(salary) AS average_salary
FROM employees;
```

What this query does: Calculates the average salary across all employees. Oracle returns the result as a number with many decimal places (like 6461.83192...). To make it cleaner, you can use ROUND: ROUND(AVG(salary), 2) shows the result with 2 decimal places. This query answers "What is the average monthly salary in the company?"

```
SELECT ROUND(AVG(salary), 2) AS avg_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees;
```

What this query does: Shows three statistics at the same time: average salary (rounded to 2 decimals), minimum salary, and maximum salary. You can use multiple aggregate functions in the same SELECT. Each function calculates independently across all rows. This is very useful for a quick summary of data.

```
SELECT ROUND(AVG(salary), 2) AS avg_sales_salary
FROM employees
```

```
WHERE department_id = 80;
```

What this query does: Calculates the average salary for employees in department 80 (Sales). By adding a WHERE clause, you can calculate the average for any subset of data. This helps compare salaries across departments: "Do sales people earn more than IT people on average?"

2.4 MIN and MAX - Minimum and Maximum

MIN returns the smallest value and MAX returns the largest value in a column. They work with numbers, dates, and text. For numbers, MIN gives the lowest number and MAX gives the highest. For dates, MIN gives the earliest date and MAX gives the latest date. For text, MIN/MAX uses alphabetical order. Both functions ignore NULL values.

```
SELECT MIN(salary) AS lowest_salary,
MAX(salary) AS highest_salary,
MAX(salary) - MIN(salary) AS salary_range
FROM employees;
```

What this query does: Finds the lowest and highest salary in the company. It also calculates the salary range (difference between max and min). The expression MAX(salary) - MIN(salary) shows you can do math with aggregate functions. The salary range tells you how wide the pay gap is.

```
SELECT MIN(hire_date) AS first_hire,
MAX(hire_date) AS last_hire
FROM employees;
```

What this query does: Finds the earliest (oldest) and latest (newest) hire date. MIN(hire_date) gives the date when the first employee was hired. MAX(hire_date) gives the date when the most recent employee was hired. This shows how long the company has been hiring. You can see the date range of all employees.

```
SELECT MIN(first_name) AS first_name_alphabetically,
MAX(first_name) AS last_name_alphabetically
FROM employees;
```

What this query does: Finds the first and last employee names in alphabetical order. MIN(first_name) returns the name that comes first alphabetically (like "Adam"). MAX(first_name) returns the name that comes last alphabetically (like "William"). This shows that MIN and MAX work with text columns too, not just numbers and dates.

2.5 Combining Multiple Aggregate Functions

You can use many aggregate functions in a single query. Each one calculates its result across all rows (or filtered rows if WHERE is used). This is like a "data summary" or "dashboard" query. Managers love these queries because they get all key numbers in one result.

```
SELECT
COUNT(*) AS total_employees,
COUNT(DISTINCT department_id) AS departments,
COUNT(DISTINCT job_id) AS job_types,
ROUND(AVG(salary), 2) AS avg_salary,
ROUND(AVG(salary + salary * NVL(commission_pct, 0)), 2) AS avg_total_pay,
SUM(salary) AS total_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees;
```

What this query does: Creates a complete summary of the employees table in one query. It shows: total number of employees, number of departments, number of job types, average salary, average total pay (with commission, using NVL to handle NULLs), total salary, minimum salary, and maximum salary. NVL(commission_pct, 0) converts NULL commissions to 0 so the calculation works correctly for everyone.

Section 3: GROUP BY

GROUP BY is one of the most powerful features in SQL. It divides rows into groups based on a column value. Then, aggregate functions calculate results for each group separately. For example, GROUP BY department_id creates one group for each department. Then COUNT, SUM, AVG, etc. calculate values for each department. Think of it like this: GROUP BY answers questions like "per department" or "per job" or "per category".

3.1 GROUP BY Single Column

The most common use of GROUP BY is with a single column. Each unique value in that column creates a group. For example, GROUP BY department_id creates one group for department 10, one group for department 20, and so on. The aggregate function then calculates a value for each group independently.

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

What this query does: Shows the number of employees in each department. GROUP BY department_id creates one group per department. COUNT(*) counts the employees in each group. ORDER BY department_id sorts the results by department number. For example, you might see: department 10 has 1 employee, department 20 has 2 employees, department 80 has 34 employees, and so on.

```
SELECT department_id,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary DESC;
```

What this query does: Calculates the average salary for each department and sorts from highest to lowest. GROUP BY department_id creates groups. AVG(salary) calculates the average for each group. ORDER BY avg_salary DESC shows departments with the highest average salary first. This answers "Which department pays the most on average?"

```
SELECT department_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary,
SUM(salary) AS total_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees
GROUP BY department_id
ORDER BY total_salary DESC;
```

What this query does: Creates a full salary report per department. For each department, it shows: number of employees, average salary, total salary (monthly payroll), minimum salary, and maximum salary. Results are sorted by total salary from high to low. This is a very common report that managers request. You can see which department has the biggest payroll and which has the widest salary range.

3.2 GROUP BY Multiple Columns

You can group by more than one column. When you use GROUP BY col1, col2, Oracle creates a group for each unique combination of col1 and col2. For example, GROUP BY department_id, job_id creates one group for department 10 + job IT_PROG, another group for department 10 + job SA_REP, and so on. This lets you analyze data at a more detailed level.

```
SELECT department_id, job_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id, job_id
ORDER BY department_id, job_id;
```

What this query does: Shows how many employees have each job in each department. GROUP BY department_id, job_id creates a group for every department + job combination. For example, you can see how many Sales Representatives (SA_REP) work in department 80, or how many IT Programmers (IT_PROG) work in department 60. This is more detailed than grouping by just department.

```
SELECT department_id, job_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department_id, job_id
ORDER BY department_id, avg_salary DESC;
```

What this query does: Shows the average salary and employee count for each job within each department. ORDER BY department_id, avg_salary DESC first sorts by department, then by average salary within each department (highest first). This helps answer questions like "In department 80, which job pays the most?"

3.3 GROUP BY with WHERE

You can use WHERE with GROUP BY. WHERE filters rows BEFORE grouping. Only rows that pass the WHERE condition are included in the groups. The order of execution is very important: FROM (get data) > WHERE (filter rows) > GROUP BY (create groups) > aggregate functions (calculate) > ORDER BY (sort). Remember: WHERE filters rows first, then GROUP BY creates groups from the remaining rows.

```
SELECT department_id, COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
```

```
WHERE salary > 10000
GROUP BY department_id
ORDER BY avg_salary DESC;
```

What this query does: First, WHERE salary > 10000 filters to keep only employees with salary above 10000. Then GROUP BY department_id groups these high-salary employees by department. COUNT and AVG calculate values for each group. The result shows how many high-salary employees each department has and their average salary. This is different from grouping all employees because we only look at employees earning more than 10000.

```
SELECT job_id, COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary,
SUM(salary) AS total_salary
FROM employees
WHERE job_id LIKE '%REP%'
GROUP BY job_id
ORDER BY num_employees DESC;
```

What this query does: First filters employees whose job_id contains "REP" (representative roles). Then groups by job_id and calculates count, average salary, and total salary for each representative type. ORDER BY num_employees DESC shows the representative type with the most employees first. This answers "What do our representatives earn?"

3.4 GROUP BY with Expressions

You can GROUP BY an expression, not just a column name. For example, you can group by year extracted from a date, or by a calculated value. When you GROUP BY an expression, the same expression must appear in the GROUP BY clause and in the SELECT list. You can also use a column alias in GROUP BY in Oracle.

```
SELECT EXTRACT(YEAR FROM hire_date) AS hire_year,
COUNT(*) AS num_hired
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY hire_year;
```

What this query does: Groups employees by the year they were hired. EXTRACT(YEAR FROM hire_date) gets just the year from the hire_date. GROUP BY groups all employees hired in the same year together. COUNT(*) shows how many employees were hired each year. This answers "How many people did we hire per year?"

```
SELECT
CASE
WHEN salary < 5000 THEN 'Low'
WHEN salary BETWEEN 5000 AND 10000 THEN 'Medium'
WHEN salary > 10000 THEN 'High'
END AS salary_level,
COUNT(*) AS num_employees
FROM employees
GROUP BY
CASE
WHEN salary < 5000 THEN 'Low'
WHEN salary BETWEEN 5000 AND 10000 THEN 'Medium'
WHEN salary > 10000 THEN 'High'
END
ORDER BY num_employees DESC;
```

What this query does: Groups employees into three salary categories (Low, Medium, High) using CASE. The same CASE expression appears in both SELECT and GROUP BY. COUNT(*) shows how many employees are in each salary level. ORDER BY num_employees DESC shows the biggest group first. This answers "How many employees earn low, medium, or high salaries?"

Section 4: HAVING - Filter Groups

HAVING filters groups AFTER they are created by GROUP BY. Think of it like WHERE, but for groups. WHERE filters individual rows before grouping. HAVING filters entire groups after aggregation. The order is important: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY. A common question is "Why do we need HAVING when we have WHERE?" The answer is: WHERE cannot use aggregate functions. If you want to filter groups by COUNT, SUM, or AVG, you MUST use HAVING.

4.1 HAVING with COUNT

The most common use of HAVING is with COUNT. You can filter groups to show only groups that have a certain number of rows. For example, show only departments with more than 5 employees, or show only jobs that have at least 3 people. HAVING COUNT(*) > 5 means "keep only groups with more than 5 rows".

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5
ORDER BY num_employees DESC;
```

What this query does: Shows only departments that have more than 5 employees. GROUP BY creates groups per department. COUNT(*) counts employees in each group. HAVING COUNT(*) > 5 removes departments with 5 or fewer employees. Without HAVING, you would see all departments. With HAVING, you see only the larger departments. This answers "Which departments have more than 5 people?"

```
SELECT job_id, COUNT(*) AS num_employees
FROM employees
GROUP BY job_id
HAVING COUNT(*) >= 2
ORDER BY num_employees DESC;
```

What this query does: Shows only job types that have 2 or more employees. Some job types might have only 1 person (like the President). HAVING COUNT(*) >= 2 removes these single-person job types. This helps you see which roles are common in the company. You can change the number in HAVING to adjust the filter.

4.2 HAVING with SUM

You can use HAVING with SUM to filter groups by their total. For example, show only departments where the total salary is greater than a certain amount. This is useful for budget analysis. Managers can see which departments spend the most on salaries.

```
SELECT department_id,
COUNT(*) AS num_employees,
SUM(salary) AS total_salary
FROM employees
GROUP BY department_id
HAVING SUM(salary) > 20000
ORDER BY total_salary DESC;
```

What this query does: Shows departments where the total monthly salary exceeds 20000. SUM(salary) calculates the total payroll for each department. HAVING SUM(salary) > 20000 filters to keep only departments whose total payroll is more than 20000. This is useful for budget analysis: "Which departments have the largest salary budgets?"

4.3 HAVING with AVG

HAVING with AVG lets you filter groups by their average value. For example, show only departments where the average salary is above a certain level. This helps identify high-paying or low-paying departments. You can also combine ROUND with AVG in HAVING for cleaner conditions.

```
SELECT department_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 10000
ORDER BY avg_salary DESC;
```

What this query does: Shows only departments where the average salary is greater than 10000. AVG(salary) calculates the average for each department. HAVING AVG(salary) > 10000 keeps only departments with an average above 10000. This answers "Which departments pay well on average?"

4.4 HAVING with WHERE (Both Together)

You can use WHERE and HAVING in the same query. WHERE filters rows first (before grouping). Then GROUP BY creates groups from the remaining rows. Then HAVING filters these groups. This two-step filtering is very powerful. Think of it as: WHERE selects the data, HAVING selects the groups.

```
SELECT department_id, job_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE department_id IN (80, 90, 100)
GROUP BY department_id, job_id
HAVING COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

What this query does: First, WHERE keeps only employees from departments 80, 90, or 100. Then GROUP BY creates groups for each department + job combination. Then HAVING keeps only groups with 2 or more employees. The result shows job types (within

these 3 departments) that have at least 2 people, along with their average salary. This is a two-step filter: filter by department, then by group size.

```
SELECT department_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE commission_pct IS NOT NULL
GROUP BY department_id
HAVING COUNT(*) > 3
ORDER BY avg_salary DESC;
```

What this query does: First, WHERE keeps only employees who have a commission (commission_pct IS NOT NULL). Then GROUP BY groups these commission-earning employees by department. Then HAVING keeps only departments that have more than 3 commission-earning employees. This answers "In which departments do more than 3 people earn commission, and what is their average salary?"

Section 5: Practice Questions

Now it is your turn to practice! Try to write the SQL queries yourself before looking at the answers. These questions cover everything from this lesson: ORDER BY, aggregate functions, GROUP BY, and HAVING. Use the Oracle HR schema tables: EMPLOYEES, DEPARTMENTS, JOBS. Good luck!

5.1 ORDER BY Practice

Q1: Show all employees ordered by annual salary (salary * 12) from high to low. Use an alias for the annual salary column.

```
-- Answer Q1
SELECT first_name, last_name, salary,
salary * 12 AS annual_salary
FROM employees
ORDER BY salary * 12 DESC;
```

Q2: Show employees ordered by last name in case-insensitive alphabetical order.

```
-- Answer Q2
SELECT first_name, last_name
FROM employees
ORDER BY LOWER(last_name);
```

Q3: Show first_name, last_name, and commission_pct. Sort by commission_pct in descending order. Use NULLS FIRST to show employees without commission first.

```
-- Answer Q3
SELECT first_name, last_name, commission_pct
FROM employees
ORDER BY commission_pct DESC NULLS FIRST;
```

5.2 Aggregate Functions Practice

Q4: Show the number of employees, average salary (rounded to 2), and total salary for ALL employees in a single query.

```
-- Answer Q4
SELECT
COUNT(*) AS total_employees,
ROUND(AVG(salary), 2) AS avg_salary,
SUM(salary) AS total_salary
FROM employees;
```

Q5: Show how many different job types exist in the company and how many different departments have employees. Use one query.

```
-- Answer Q5
SELECT
COUNT(DISTINCT job_id) AS job_types,
COUNT(DISTINCT department_id) AS departments
FROM employees;
```

Q6: Show the earliest hire date and the latest hire date from the employees table.

```
-- Answer Q6
SELECT MIN(hire_date) AS first_hire,
MAX(hire_date) AS last_hire
FROM employees;
```

Q7: Show the average salary for the Sales department (department_id = 80). Round the result to 2 decimal places.

-- Answer Q7

```
SELECT ROUND(AVG(salary), 2) AS avg_sales_salary
FROM employees
WHERE department_id = 80;
```

5.3 GROUP BY Practice

Q8: Show the number of employees in each department. Order by department_id.

-- Answer Q8

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
ORDER BY department_id;
```

Q9: Show the total salary (SUM) and average salary (AVG, rounded to 2) for each job_id. Order by total salary from high to low.

-- Answer Q9

```
SELECT job_id,
SUM(salary) AS total_salary,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY job_id
ORDER BY total_salary DESC;
```

Q10: Show how many employees were hired in each year (use EXTRACT). Order by year.

-- Answer Q10

```
SELECT EXTRACT(YEAR FROM hire_date) AS hire_year,
COUNT(*) AS num_hired
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY hire_year;
```

Q11: Show department_id, job_id, and employee count for groups that have 2 or more employees. Order by department_id and job_id.

-- Answer Q11

```
SELECT department_id, job_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id, job_id
HAVING COUNT(*) >= 2
ORDER BY department_id, job_id;
```

5.4 HAVING Practice

Q12: Show departments where the average salary is greater than 8000. Include the employee count and average salary in the result.

-- Answer Q12

```
SELECT department_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 8000
ORDER BY avg_salary DESC;
```

Q13: Show departments where the total monthly salary (SUM) is greater than 30000.

-- Answer Q13

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id
HAVING SUM(salary) > 30000
ORDER BY total_salary DESC;
```

Q14: Show job_id and employee count for jobs that have more than 3 employees. Only include employees who earn more than 5000 (use WHERE).

-- Answer Q14

```
SELECT job_id, COUNT(*) AS num_employees
FROM employees
WHERE salary > 5000
GROUP BY job_id
HAVING COUNT(*) > 3
```

```
ORDER BY num_employees DESC;
```

Q15: For employees hired after 2004, show how many employees each department has. Only show departments with 3 or more employees from this group.

```
-- Answer Q15
SELECT department_id, COUNT(*) AS num_employees
FROM employees
WHERE EXTRACT(YEAR FROM hire_date) > 2004
GROUP BY department_id
HAVING COUNT(*) >= 3
ORDER BY num_employees DESC;
```

5.5 Combined Practice

Q16: Show department_id and the salary range (MAX - MIN salary) for each department. Only show departments where the salary range is greater than 5000.

```
-- Answer Q16
SELECT department_id,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary,
MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY department_id
HAVING MAX(salary) - MIN(salary) > 5000
ORDER BY salary_range DESC;
```

Q17: Show a salary summary per job: count, average (rounded), total, min, and max. Only show jobs where the average salary is at least 8000. Sort by average descending.

```
-- Answer Q17
SELECT job_id,
COUNT(*) AS num_employees,
ROUND(AVG(salary), 2) AS avg_salary,
SUM(salary) AS total_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees
GROUP BY job_id
HAVING AVG(salary) >= 8000
ORDER BY avg_salary DESC;
```

Q18: Show the number of employees whose last name contains "an" per department. Only show departments where at least 2 such employees exist.

```
-- Answer Q18
SELECT department_id, COUNT(*) AS num_employees
FROM employees
WHERE last_name LIKE '%an%'
GROUP BY department_id
HAVING COUNT(*) >= 2
ORDER BY num_employees DESC;
```

Section 6: Quick Reference

This table gives you a quick summary of everything in this lesson. Use it to review and remember what each function and clause does. Keep this table nearby when you practice.

Keyword / Function	Purpose	Example
ORDER BY col DESC	Sort descending (high to low)	ORDER BY salary DESC
ORDER BY 1, 3 DESC	Sort by column position numbers	ORDER BY 1, 3 DESC
ORDER BY expression	Sort by calculation or function	ORDER BY salary * 12
NULLS FIRST / NULLS LAST	Control where NULL values appear in sort	ORDER BY comm DESC NULLS FIRST
COUNT(*)	Count ALL rows (including NULLs)	SELECT COUNT(*) FROM t
COUNT(col)	Count non-NULL values in column	COUNT(commission_pct)
COUNT(DISTINCT col)	Count unique non-NULL values	COUNT(DISTINCT dept_id)

SUM(col)	Total (sum) of values	SUM(salary)
AVG(col)	Average (mean) of values	ROUND(AVG(salary), 2)
MIN(col)	Smallest value	MIN(salary)
MAX(col)	Largest value	MAX(hire_date)
GROUP BY col	Group rows by column value	GROUP BY department_id
HAVING condition	Filter groups after aggregation	HAVING COUNT(*) > 5
NVL(col, value)	Replace NULL with a default value	NVL(comm, 0)

SQL Query Execution Order

Understanding the execution order of SQL clauses is very important. Oracle processes a query in this order: (1) FROM - get data from tables, (2) WHERE - filter rows, (3) GROUP BY - create groups, (4) HAVING - filter groups, (5) SELECT - choose columns and calculate aggregates, (6) ORDER BY - sort the result. This order explains why WHERE cannot use aggregate functions (aggregates are calculated after WHERE) and why HAVING can (aggregates are calculated before HAVING).

Step	Clause	What Happens
1	FROM	Load data from table(s)
2	WHERE	Filter individual rows (no aggregates allowed)
3	GROUP BY	Create groups based on column values
4	HAVING	Filter groups (aggregates allowed here)
5	SELECT	Choose columns and calculate aggregate results
6	ORDER BY	Sort the final result

WHERE vs HAVING - Key Differences

Feature	WHERE	HAVING
When it runs	Before GROUP BY (filters rows)	After GROUP BY (filters groups)
What it filters	Individual rows	Entire groups
Aggregate functions	NOT allowed	Allowed (COUNT, SUM, AVG...)
Example	WHERE salary > 5000	HAVING COUNT(*) > 3

Congratulations on completing Lesson 4! You now know how to analyze data using aggregate functions, group data with GROUP BY, filter groups with HAVING, and sort results with advanced ORDER BY. These are the most important tools for data analysis in SQL. Practice these queries every day. In the next lesson, you will learn about JOINs, which let you combine data from multiple tables. Keep practicing, DTank54!